

Universidade Estadual da Paraíba
Ciência da Computação
Tópicos Especiais em Computação
Docente: Dra. Sabrina de Figueirêdo Souto

Vitor Francisco Farias de Souza
Nidlan Hendell de Araújo Farias
Thales Henrique de Souza Teles
Rafael Trajano do Nascimento

ManualBot

Uma conversa prática sobre arduinos

Campina Grande - PB
07/08/2026

SUMÁRIO

1. Introdução....3
2. Objetivos....3
3. Fundamentação teórica....3
3.1 Engenharia de Software....3
3.2 Large Language Models (LLMs)....4
3.3 Banco Vetorial....4
3.4 APIs (Application Programming Interfaces)....4
3.5 Frameworks Utilizados....4
4. Desenvolvimento....5
4.1 Arquitetura....5
4.2. Pipeline de Ingestão e Indexação (Offline)....5
4.3 Pipeline de Consulta RAG (Online)....6
4.4 Tecnologias Utilizadas....6
4.5 Implementação....7
5. Resultados....8
6. Discussão....10
6.1 Pontos Positivos....10
6.2 Dificuldades Encontradas e Limitações....10
6.3 Possíveis Melhorias....11
7. Conclusão....11
8. Referências....12

1. Introdução

A documentação técnica e os manuais de componentes eletrônicos frequentemente utilizam um vocabulário altamente especializado e denso. Para iniciantes ou usuários sem formação aprofundada na área, a interpretação correta dessas especificações técnicas representa uma barreira de entrada significativa, levando a erros de montagem, desperdício de tempo e subaproveitamento dos recursos dos componentes.

Embora as informações necessárias para o uso correto dos equipamentos estejam disponíveis nos manuais, o volume de dados e a linguagem estritamente técnica dificultam a busca rápida e precisa por respostas pontuais. As ferramentas tradicionais de busca por palavras-chave muitas vezes retornam trechos genéricos ou exigem uma leitura extensiva do documento para solucionar dúvidas simples de configuração e operação.

O objetivo deste projeto é solucionar estas problemáticas apresentadas, desenvolvendo um assistente inteligente baseado na arquitetura *Retrieval-Augmented Generation* (RAG) capaz de responder a dúvidas em linguagem natural sobre manuais técnicos de eletrônica (com foco inicial na ecologia Arduino). O sistema busca automatizar a recuperação de trechos relevantes da documentação técnica e fornecê-los como contexto a um Modelo de Linguagem de Grande Porte (LLM), gerando respostas precisas, simplificadas e devidamente fundamentadas com a citação das fontes originais.

2. Objetivos

O projeto tem como objetivo aplicar conceitos de Embeddings, Busca Semântica, Bancos Vetoriais, RAG, Prompt Engineering, Workflows e LLMs, demonstrando como construir assistentes inteligentes especializados em documentação técnica.

Desenvolver uma aplicação interativa e intuitiva em Python utilizando a biblioteca Streamlit, com o propósito de fornecer uma interface amigável para consulta em linguagem natural à documentação técnica do microcontrolador ESP32.

Integrar os componentes da arquitetura RAG (*Retrieval-Augmented Generation*), conectando a extração de documentos em PDF via PyMuPDF, o banco vetorial ChromaDB, o orquestrador de fluxos n8n e a API do modelo de linguagem Google Gemini.

Implementar o pipeline completo de RAG, abrangendo as etapas de leitura e extração de texto, fragmentação inteligente dos documentos (*chunking*), geração de *embeddings*, busca semântica por similaridade e engenharia de *prompts* orientada à prevenção de alucinações.

Validar experimentalmente a precisão e a confiabilidade das respostas geradas pelo assistente através de testes com perguntas técnicas de controle, avaliando o tempo de resposta, a relevância dos trechos recuperados do banco vetorial e a aderência das respostas às fontes originais

3. Fundamentação teórica

3.1 Engenharia de Software

A Engenharia de Software fornece a sustentação disciplinar para a construção do sistema, aplicando boas práticas de ciclo de vida de desenvolvimento, versionamento de código, separação de responsabilidades e princípios de modularidade. Através dela,

garante-se que a solução seja escalável, legível para manutenção em equipe e devidamente documentada desde a ingestão dos dados até a entrega da interface final.

3.2 Large Language Models (LLMs)

Os Modelos de Linguagem de Larga Escala são redes neurais avançadas treinadas em volumes massivos de dados textuais para compreender, sintetizar e gerar texto contextualizado. No ManualBot, a LLM atua como o "motor de raciocínio e síntese". Em vez de atuar como uma fonte primária e irrestrita de conhecimento, o que poderia gerar "alucinações", a LLM é utilizada dentro de uma arquitetura controlada para ler o contexto técnico recuperado dos manuais e formular uma resposta clara, objetiva e fundamentada para o usuário.

3.3 Banco Vetorial

O Banco Vetorial é a estrutura de armazenamento especializada em lidar com dados não estruturados representados em forma de vetores matemáticos (embeddings). Diferente dos bancos relacionais SQL (que buscam por correspondência exata de palavras-chave), o banco vetorial realiza a **busca por similaridade semântica**. Ele calcula a distância matemática (como a similaridade de cosseno) entre o vetor da pergunta do usuário e os vetores dos trechos do manual, retornando em milissegundos as seções mais relevantes do documento. Utilizado para “alimentar” a LLM

3.4 APIs (Application Programming Interfaces)

As APIs são as pontes de comunicação padronizadas que permitem a interoperabilidade entre os diferentes serviços do ecossistema do projeto. O ManualBot utiliza APIs REST para realizar a integração entre a interface de usuário, o orquestrador de fluxos (n8n) e o modelo de linguagem na nuvem (Google Gemini API). Isso garante que os dados trafeguem de forma segura e estruturada (via JSON) entre os módulos da aplicação.

3.5 Frameworks Utilizados

LangChain: Framework especializado no desenvolvimento de aplicações alimentadas por LLMs. É utilizado para abstrair e encadear o pipeline de RAG, gerenciando o fatiamento dos textos (*chunking*), a injeção de metadados e a integração nativa com o banco vetorial e com os modelos de linguagem.

Streamlit: Framework em Python voltado para a criação ágil de interfaces web interativas para projetos de ciência de dados e IA. Ele permite transformar dados e chamadas de API em uma interface visual fluida para o usuário final com poucas linhas de código.

n8n: Framework/Plataforma de orquestração de workflows baseada em nós. Atua como o backend do sistema, gerenciando a lógica de negócios, a chamada aos endpoints de busca semântica e a estruturação do prompt enviado à LLM.

4. Desenvolvimento

4.1 Arquitetura

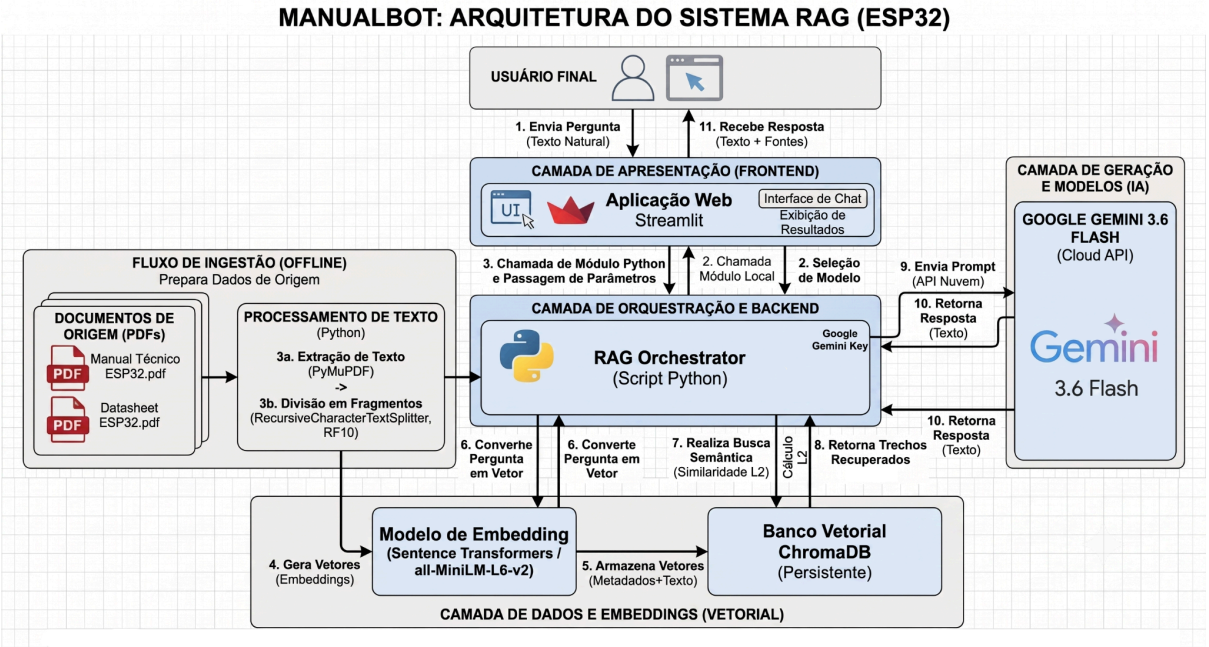


Figura 1: Arquitetura do sistema. **Fonte:** Elaborada pelos autores com o auxílio da IA Google Gemini (2026).

O ManualBot foi projetado sob uma arquitetura modular em camadas, aplicando o padrão RAG (*Retrieval-Augmented Generation*) para a consulta inteligente à documentação técnica do microcontrolador ESP32.

O ciclo de vida de processamento do sistema é dividido em duas etapas operacionais complementares: o **Processamento Offline** (preparação da base) e o **Processamento Online** (atendimento da consulta em tempo real).

4.2. Pipeline de Ingestão e Indexação (Offline)

1. **Carregamento dos Documentos:** Os manuais técnicos em formato PDF são lidos a partir do diretório de origem.
2. **Extração de Texto:** O módulo em Python utiliza a biblioteca **PyMuPDF** para realizar a leitura das páginas e extração limpa do texto contido nos manuais.
3. **Chunking (Divisão em Fragmentos):** Aplica-se o algoritmo RecursiveCharacterTextSplitter para fatiar os textos longos em trechos menores e semanticamente coesos, associando a cada *chunk* os seus respectivos metadados (como nome do arquivo e número da página).
4. **Geração de Embeddings e Persistência:** Os *chunks* de texto são processados pelo modelo de *embedding* Sentence Transformers (all-MiniLM-L6-v2), que gera os vetores numéricos correspondentes. Esses vetores, juntamente com o texto puro e os metadados, são salvos e indexados no banco de dados vetorial **ChromaDB**.

4.3 Pipeline de Consulta RAG (Online)

1. **Entrada do Usuário:** O usuário final digita uma pergunta em linguagem natural na interface web construída com **Streamlit** (passo 1).
2. **Orquestração e Conversão em Vetor:** A pergunta é encaminhada para o script principal em Python (RAG Orchestrator), que envia a string da pergunta para o modelo de *embedding* para convertê-la em um vetor equivalente (passos 3 e 6).
3. **Busca Semântica:** O LangChain realiza uma busca por similaridade (utilizando cálculo de distância L2/Cosseno) no **ChromaDB** (passo 7).
4. **Recuperação do Contexto:** O ChromaDB retorna os k's trechos mais relevantes do manual acompanhados dos seus metadados (requisito RF07) (passo 8).
5. **Construção do Prompt e Invocação da LLM:** O LangChain injeta os trechos recuperados e a pergunta original dentro de um *prompt* estruturado com regras restritivas (prevenindo alucinações) e o envia para o modelo selecionado:
    - **Fluxo Nuvem (9a/10a):** Envia a requisição via API para o **Google Gemini Pro** usando a Google Gemini Key.
6. **Exibição do Resultado:** A LLM gera a resposta fundamentada. O orquestrador recebe o texto final e o envia para o **Streamlit**, que renderiza na tela a resposta amigável acompanhada da citação dos documentos, páginas e trechos utilizados (passos 11 e RF10).

4.4 Tecnologias Utilizadas

Conceito	**Tecnologia utilizada**
Linguagem	Python
Framework	LangChain
Banco vetorial	ChromaDB
LLM	Gemini 3.6 flash
Extração de PDF	PyMuPDF
Interface	StreamLit
WorkFlow	n8n
Versionamento	Git
Embeddings	Sentence Transformers

Tabela 1: Stack Tecnológica.
Fonte: Autor.

4.5 Implementação

Para garantir a sustentabilidade, modularidade e clareza no ciclo de vida do software, o **ManualBot** foi projetado com uma clara separação de responsabilidades entre suas camadas funcionais. O sistema é sustentado por cinco componentes centrais que atuam em sinergia desde a ingestão da base documental até a entrega da resposta ao usuário final.

O ponto de partida da aplicação é o **Módulo de Dados**, responsável pela recepção e persistência da base documental do projeto. Ele armazena os arquivos técnicos em formato PDF na pasta local (/data) e gerencia o **ChromaDB**, que atua como o banco vetorial persistente da solução. Este banco guarda não apenas as representações vetoriais (*embeddings*) do texto, mas também os metadados associados a cada fragmento, como o nome do arquivo original e o número da página.

A transformação dos dados brutos em informação estruturada fica a encargo do **Módulo de Processamento (Ingestão e Chunking)**. Atuando como um pipeline de ETL (*Extract, Transform, Load*), este módulo utiliza a biblioteca **PyMuPDF** para realizar a leitura limpa do texto dos manuais. Em seguida, aplica o algoritmo RecursiveCharacterTextSplitter para fatiar o conteúdo em *chunks* de 800 caracteres (com sobreposição de 150 caracteres). Essa estratégia garante que os blocos de texto mantenham coesão semântica e preservem as referências das páginas originais antes da indexação.

A camada matemática e generativa reside no **Módulo de Inteligência Artificial**, que se divide em duas atribuições principais. A primeira é a vetorização, executada pelo modelo Sentence Transformers (all-MiniLM-L6-v2), responsável por converter as perguntas e os trechos de texto em vetores para o cálculo de similaridade semântica. A segunda é a geração de linguagem natural propriamente dita, que conecta os motores de síntese de texto, permitindo a comunicação tanto via API na nuvem com o **Google Gemini Pro**.

Toda essa complexidade é gerenciada pelo **Módulo de Integração e Orquestração (RAG Orchestrator)**, que funciona como o backend central da aplicação. Ele recebe a requisição do usuário, aciona a vetorização da pergunta, realiza a busca por similaridade L2 no ChromaDB e aplica as técnicas de *Prompt Engineering* para injetar as diretrizes de restrição anti-alucinação na LLM escolhida.

Por fim, o **Módulo de Interface (Frontend)**, desenvolvido em **Streamlit**, provê a camada de apresentação visual intuitiva para o usuário. Ele oferece um campo para digitação das dúvidas em linguagem natural, o seletor dinâmico de modelo (*Model Switcher*) para alternar entre execução em nuvem ou local, e o painel de resultados que exibe a resposta final e destaca as fontes, páginas e trechos consultados, garantindo o cumprimento integral dos requisitos **RF06**, **RF10** e **RF11** do projeto.

5. Resultados

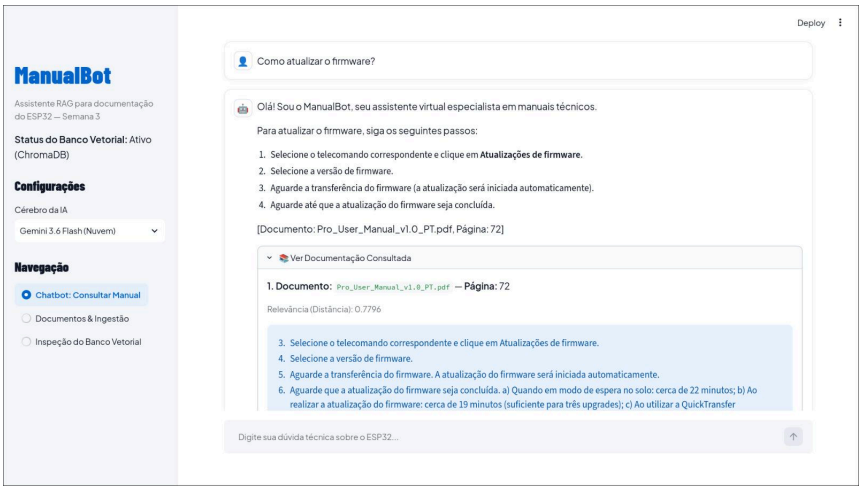


Figura 2: Tela principal, onde o usuário irá realizar suas perguntas e consequentemente ter sua resposta
Fonte: Autor

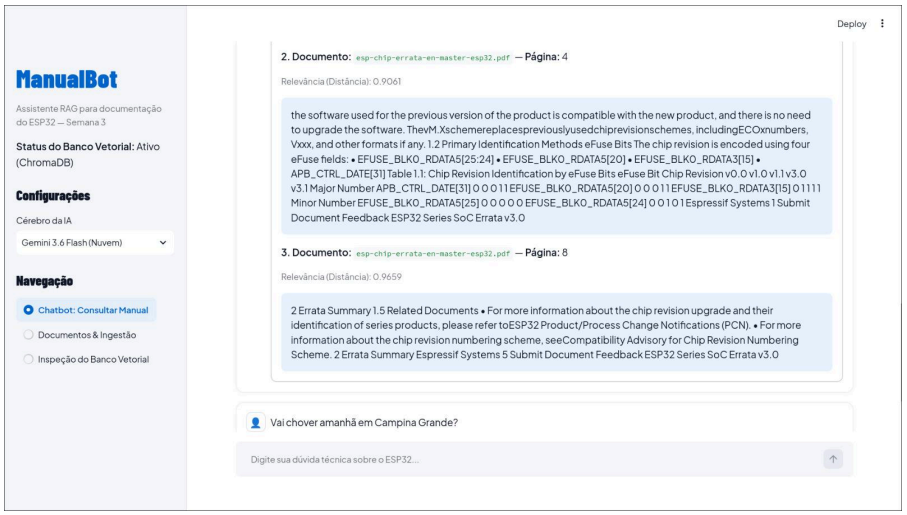


Figura 3: Retorno de trechos retirados dos manuais
Fonte: Autor

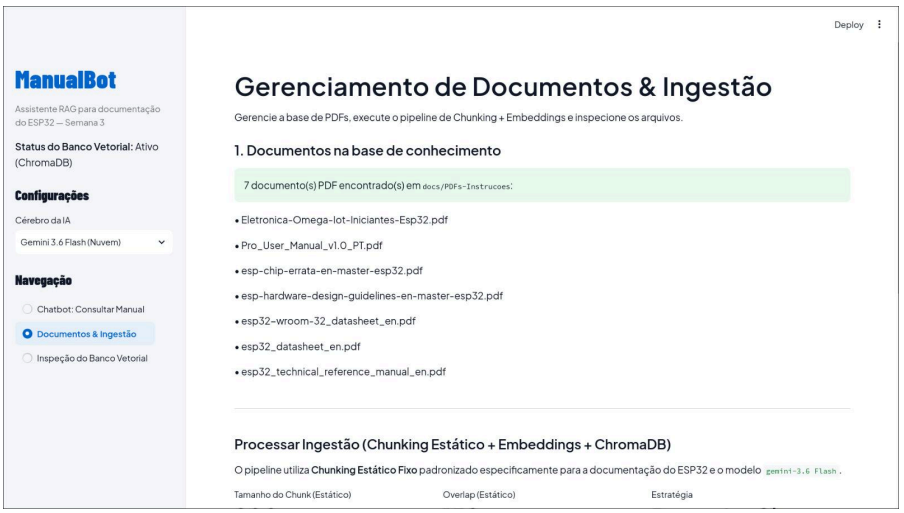


Figura 4: Tela de gerenciamento dos documentos da base documental

Fonte: Autor.



Figura 5: Tela de visualização de trechos(chunks) retirados da base documental

Fonte: Autor.

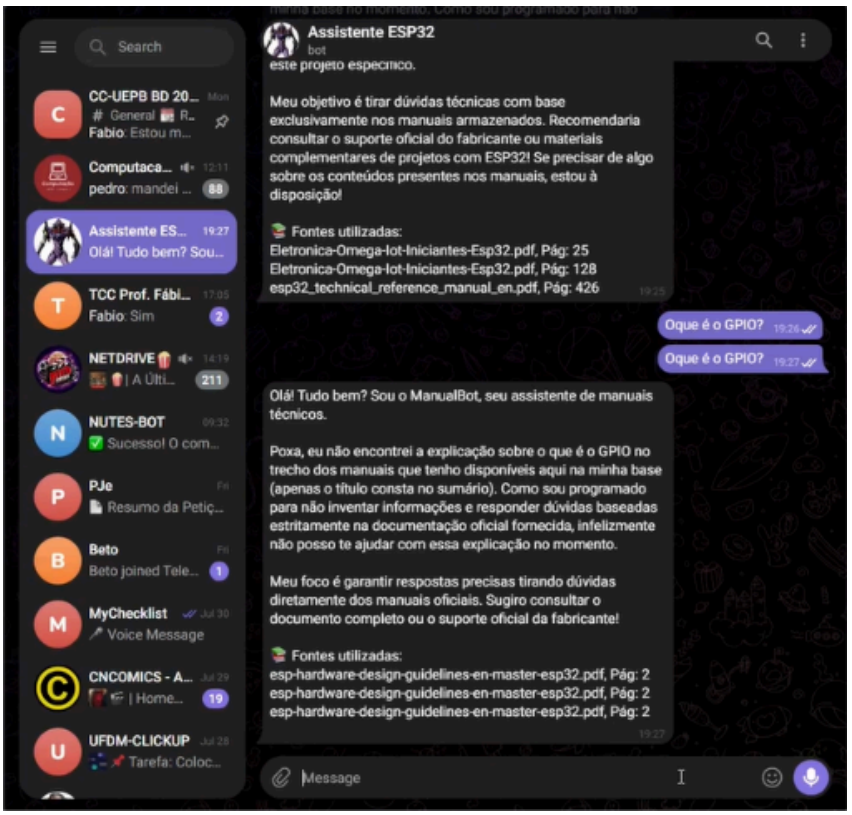


Figura 6: Interação do usuário com o chat via Telegram

Fonte: Autor.

6. Discussão

6.1 Pontos Positivos

O projeto obteve grande êxito na construção de uma arquitetura modular e escalável. O principal ponto positivo é a fidelidade das respostas, garantida pela regra de restrição implementada no system prompt (RF11), que diminui drasticamente o risco de alucinações do modelo (LLM). Outro diferencial foi a adoção de um modelo de embeddings multilíngue (paraphrase-multilingual-MiniLM-L12-v2), que permitiu a realização de buscas semânticas cruzadas: o usuário realiza perguntas em Português e o sistema recupera com precisão trechos de manuais originais em Inglês. Além disso, a separação do core do RAG através de uma API REST (FastAPI) permitiu que a solução servisse múltiplos clientes simultaneamente, como a interface rica e dinâmica no Streamlit e a integração de automação omnichannel via n8n (Telegram).

6.2 Dificuldades Encontradas e Limitações

Estrutura Visual e Tabelas nos PDFs: A biblioteca PyMuPDF realiza uma extração eficiente de texto corrido, mas apresentou dificuldades ao ler tabelas complexas de pinagem (pinouts) e esquemáticos elétricos do ESP32. Quando os dados de uma tabela eram convertidos para texto puro, a perda da estrutura tridimensional de linhas e colunas

ocasionalmente prejudicava a precisão da busca semântica para consultas sobre pinos específicos.

Embora a aplicação seja autossuficiente na vetorização e busca (rodando 100% via CPU local com o ChromaDB), ela atualmente depende de conexão à internet para a etapa de geração final da resposta, pois utiliza a API do Google Gemini 3.6 Flash. Outra limitação é a perda de elementos visuais: o processo de chunking com PyMuPDF e RecursiveCharacterTextSplitter foca inteiramente na extração de texto. Logo, esquemáticos elétricos, diagramas de pinagem e gráficos vitais para a documentação técnica de hardware não são entregues ao LLM, restringindo a capacidade de explicação visual.

6.3 Possíveis Melhorias

Para o futuro da aplicação, recomenda-se a implementação de Análise Multimodal (Visão Computacional), permitindo que o sistema extraia imagens e diagramas dos PDFs e os submeta a LLMs visuais.

Outra melhoria de grande impacto seria a criação de um módulo de Upload Dinâmico no front-end do Streamlit, abolindo a necessidade do usuário interagir com as pastas do sistema operacional para adicionar novos manuais.

Por fim, a conclusão da integração da infraestrutura com o Ollama para a execução de modelos locais (como o Llama 3) transformaria o ManualBot em um produto 100% autônomo, privado, offline e ideal para ambientes industriais sem acesso à internet.

7. Conclusão

O projeto ManualBot cumpriu com êxito a proposta de desenvolver um assistente inteligente capaz de responder a dúvidas em linguagem natural sobre documentações técnicas complexas, utilizando como domínio os manuais e datasheets do microcontrolador ESP32. A implementação de uma arquitetura baseada em Retrieval-Augmented Generation (RAG) provou ser uma solução eficiente e robusta para democratizar o acesso a informações técnicas sem abdicar da precisão necessária em projetos de engenharia.

Os requisitos obrigatórios definidos no escopo do projeto foram plenamente atendidos. O sistema foi capaz de realizar a extração limpa dos textos em PDF, executar o fatiamento inteligente (chunking) preservando a coesão das páginas e gerar a indexação vetorial no ChromaDB. A integração do pipeline com a API do Google Gemini Pro garantiu a geração de respostas estritamente fundamentadas e livres de alucinações (cumprindo os requisitos RF11 e RF09). Por fim, a interface em Streamlit entregou uma experiência fluida ao usuário, exibindo de forma transparente não apenas as respostas, mas também os documentos, páginas e trechos de origem que embasam cada afirmação (requisitos RF06 e RF10).

A principal contribuição do projeto reside no desenvolvimento de uma aplicação prática, modular e reproduzível de IA Generativa orientada a dados privados/específicos. O ManualBot demonstra como a combinação entre busca semântica e modelos de linguagem pode reduzir drasticamente o tempo gasto por estudantes, projetistas e engenheiros na busca

manual por especificações de registradores, pinagens e modos de operação (como o Deep Sleep) em documentos de centenas de páginas.

A execução do projeto proporcionou um valioso aprendizado prático que transcende a teoria de sala de aula. O grupo consolidou conhecimentos essenciais em Engenharia de Software aplicada à Inteligência Artificial, abrangendo desde a manipulação de dados não estruturados (PDFs) e engenharia de prompts até o entendimento profundo de embeddings, métricas de similaridade vetorial (como a distância L2) e o ecossistema do LangChain. Adicionalmente, compreendeu-se na prática a relevância de tratar os trade-offs de sistemas reais, como o equilíbrio entre o tamanho dos chunks, o consumo de tokens da API e a latência de processamento em modelos locais.

8. Referências

[1] ALAMMAR, Jay; GROOTENDORST, Maarten. Hands-On Large Language Models. 1. ed. Sebastopol: O'Reilly Media, 2024.

[2]**LANGCHAIN**. *LangChain Documentation*. [S. l.]: LangChain, Inc., 2026. Disponível em: https://python.langchain.com/docs/. Acesso em: 7 ago. 2026.

[3] STREAMLIT. *Streamlit Documentation*. [S. l.]: Snowflake Inc., 2026. Disponível em: https://docs.streamlit.io/. Acesso em: 7 ago. 2026.